Lab4: Computer Architectuur

Stijn Verhaeghe & Andres Bosman

March 2026

1 Introduction

In dit labo gaan we eerst onderzoeken wat de invloed is van Coalesced memory access, dit wil zeggen dat het best is om de threads opeenvolgend uit te laten voeren. Aan de hand van de eerste oefening wordt er onderzocht welke invloed dit heeft. Verder wordt er ook stilgestaan bij de memory types die er allemaal bestaan. Tot nu toe hebben we vooral gebruikgemaakt van de globale memory. Maar er bestaat ook bijvoorbeeld shared en constant memory. In oefening 2 wordt de invloed hiervan bekeken.

2 Oefening 1

2.1 Theoretische Achtergrond

In deze oefening wordt de impact van *coalesced memory access* op de GPU-prestaties onderzocht. Coalescing treedt op wanneer threads binnen een warp van 32 threads gelijktijdig opeenvolgende adressen in het globale geheugen aanspreken. De hardware kan deze individuele aanvragen combineren tot een minimaal aantal geheugentransacties, meestal in groepen van 32, 64 of 128 bytes.

Indien de toegang niet coalesced is, moet de DRAM-toegang sequentieel per verzoek worden afgehandeld, wat de effectieve bandbreedte drastisch verlaagt en de latency verhoogt.

2.2 Methodologie

Er zijn twee kernels ontworpen die werken op een vierkante matrix A van grootte $N \times N$:

- **Copy Kernel ($B = A$)**: Kopieert de inputmatrix naar een andere matrix.
- **Transpose Kernel ($B = A^T$)**: Transponeert de inputmatrix.

De prestaties worden geëvalueerd door de effectieve bandbreedte (BW) te berekenen:

$$BW = \frac{\text{Totaal aantal overgedragen bytes}}{\text{tijd}} \text{ (in GB/s)} \tag{1}$$

2.3 Analyse van de Resultaten

Op basis van de verzamelde data in Figuur 1 en 2 kunnen de volgende conclusies worden getrokken:

- **Toegangspatronen**: Bij de `matrixCopy` kernel lezen en schrijven opeenvolgende threads naar opeenvolgende geheugenlocaties. Dit resulteert in optimale coalescing, wat de hoge bandbreedtepiek in Figuur 2 verklaart.
- **Impact van Transpositie**: Bij de `matrixTranspose` kernel zorgt het algoritme ervoor dat threads ofwel niet-opeenvolgend lezen, ofwel niet-opeenvolgend schrijven. Dit doorbreekt het coalescing-principe, waardoor de DRAM de data niet als een efficiënte *burst* kan leveren. Dit uit zich in een lagere effectieve bandbreedte vergeleken met de copy bewerking.
- **GPU vs. CPU**: Zoals te zien in Figuur 1, is de CPU sneller voor zeer kleine matrices vanwege de overhead van geheugenallocatie en data-overdracht. Bij grotere matrices ($N > 128$) neemt de GPU de overhand door de massale parallelle verwerking.



Figure 1: Vergelijking van de uitvoeringstijd tussen CPU en GPU voor Copy en Transpose operaties.

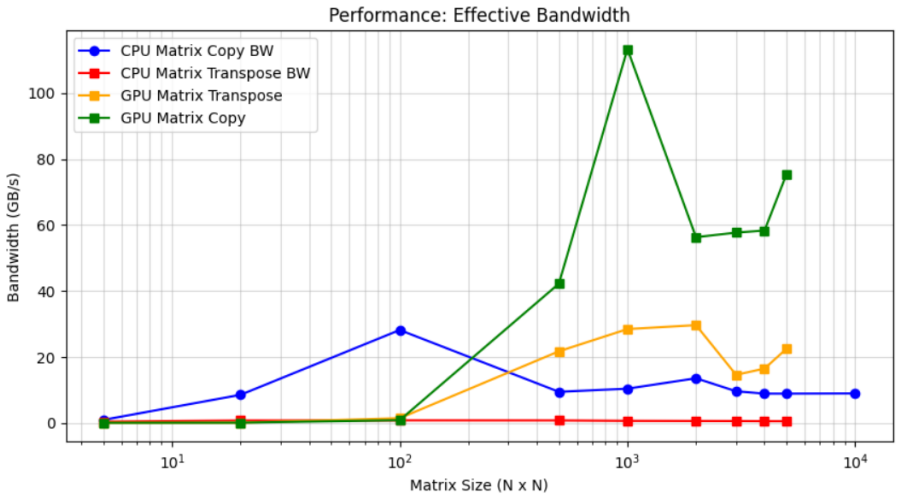


Figure 2: Vergelijking van de effectieve bandbreedte tussen CPU en GPU voor Copy en Transpose operaties.

3 Oefening 2: Global, Shared, en Constant Memory

3.1 Theoretische Achtergrond

De GPU beschikt over verschillende geheugenhiërarchieën met uiteenlopende eigenschappen van latency en bereik:

- **Global Memory**: Off-chip, hoge latency, maar toegankelijk voor alle threads en de CPU.
- **Shared Memory**: On-chip, zeer lage latency, gedeeld binnen een thread block.
- **Constant Memory**: Off-chip maar gecachet, geoptimaliseerd voor *read-only* toegang door alle threads. De grootte is beperkt tot 64 KB.

3.2 Methodologie

De verschillende geheugens worden geëvalueerd op basis van een vierkante matrix A van grootte $N \times N$:

- **Multiply Kernel** ($C = A * B$): vermenigvuldigt matrix A met matrix B en kopieert het in matrix C.

3.3 Analyse van de Resultaten

Op basis van de verzamelde data in Figuur 3 kunnen de volgende conclusies worden getrokken:

- **Constant Memory Limiet**: De resultaten tonen aan dat de *Constant Memory* kernel enkel functioneert bij kleine matrices. Vanaf $N = 129$ kan er niet meer met dit geheugen gewerkt worden. Dit bevestigt de theoretische limiet van 64 KB voor constant geheugen op de GPU. Aangezien een matrix van 129×129 deze capaciteit overschrijdt, kan de data niet langer in dit geheugen opgeslagen worden.

- **Global vs. Shared Memory**: Hoewel *Shared Memory* zich *on-chip* bevindt en een zeer lage latency heeft, liggen de uitvoeringstijden zeer dicht bij die van het *Global Memory*. Dit is te verklaren door de benodigde *overhead*: data moet eerst expliciet door de threads gemigreerd worden van het globale naar het shared geheugen. Daarnaast is het gebruik van `__syncthreads()` cruciaal om data-integriteit te garanderen, wat een extra synchronisatiekost met zich meebrengt. Bij deze specifieke matrixgrootte weegt de winst in toegangssnelheid nauwelijks op tegen deze migratiekosten.

- **Schaalbaarheid en CPU-beperkingen**: De CPU-uitvoeringstijd stijgt exponentieel naarmate N toeneemt. Dit komt door de sequentiële aard van de standaard matrixvermenigvuldiging. De GPU-kernels schalen veel efficiënter dankzij de massale parallellisatie over duizenden threads in de grid. Bij kleine matrices is de CPU echter competitief of zelfs sneller door de afwezigheid van de overhead (dataoverdracht van host naar device).

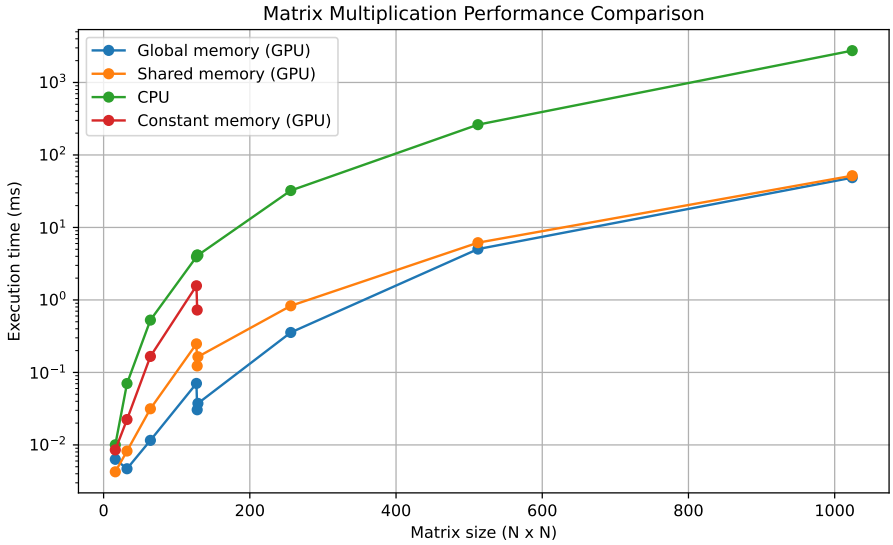


Figure 3: Matrix Multiplication Performance vergelijking

4 Antwoord vragen: Oefening 2

4.1 How much time does memory migration take (CPU to GPU global; GPU global to GPU shared)?

Deze metingen zijn niet uitgevoerd omdat de theoretische verwachting volstaat: de migratie van de CPU naar het globale GPU-geheugen duurt het langst omdat de geheugens fysiek verder uit elkaar liggen. De stap van het globale naar het shared GPU-geheugen verloopt veel sneller, omdat dit geheugen *on-chip* is en de afstand dus minimaal is.

4.2 How do the execution times of matrix multiplication compare when employing the three different memory types?

Zie deel 3.3 Analyse van de Resultaten.

4.3 How do the times scale with input and/or grid size?

De schaling van de uitvoeringstijd vertoont een drastisch verschil tussen de CPU en de GPU naarmate de matrixgrootte (N) toeneemt. Terwijl de CPU tijd stijgt omdat berekeningen sequentieel worden uitgevoerd, blijft de GPU veel efficiënter door duizenden berekeningen tegelijkertijd te verwerken.

Bij de specifieke grootte $N = 128$ profiteert de GPU van een optimale aansluiting op de hardware-architectuur. Omdat 128 een macht van twee is, vallen de groepen van 32 threads precies samen met de geheugenblokken in het DRAM. Dit resulteert in *coalescing*, waarbij de hardware alle benodigde data in minimale transacties kan ophalen. Dit maximaliseert de effectieve bandbreedte en zorgt voor een significante daling in de uitvoeringstijd per element vergeleken met de sequentiële verwerking op de CPU.

5 Code

5.1 Oefening 1

```
#include <iostream>
#include <cuda_runtime.h>
#include <vector>
//nu met meerdere blocks
// CUDA kernel to rev 1 arrays
__global__ void matrixCopyGPU(int *a, int *c, int size) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if(idx < size)
        c[idx] = a[idx];
}

__global__ void matrixTransposeGPU(int *a, int *c, int size)
    {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int invidx = ;
    if(idx < size)
        int R = rows;
        int K = columns;
        int index, invIndex;


}


void matrixCopyCPU(const std::vector<int>& matrix,
                   std::vector<int>& result) {
    result = matrix;
}
void printMatrix(const std::vector<int>& m, int rows, int
    cols) {
    for(int r = 0; r < rows; r++) {
        for(int c = 0; c < cols; c++) {
            std::cout << m[r * cols + c] << "␣";
        }
        std::cout << std::endl;
    }
```

```
35 }
36 void matrixTransposeCPU(const std::vector<int>& matrix,
37                         std::vector<int>& result, int rows, int
                               columns) {
38     int R = rows;
39     int K = columns;
40     int index, invIndex;
41     for(int row = 0; row<R;row++){
42         for(int column = 0; column < K; column++ ){
43             index= row * R + column;
44             invIndex = column * K + row;
45             result[invIndex] = matrix[index];
46         }
47     }
48 }
49 int main() {
50     int rows = 3;
51     int columns = 3;
52     int size = rows * columns;
53     std::vector<int> matrix(size);
54     matrix = {1,4,2,3,6,8,7,5,9};
55     std::vector<int> result_matrix(size);
56
57     if(rows != columns){// bij rechthoek zou kolom/rij
           omgedraait moeten worden
58         std::cout << "rechthoek␣kan␣nog␣niet" << std::endl;
               return 1;
59     }
60
61     int* d_matrix; //d_ = device = gpu
62     int* d_result_matrix;
63
64     int threads = 256;
65     int blocks = (matrix.size() + threads - 1) / threads;
66 /*
67 -------------------------begin cpu
       -------------------------------------
68 */
69 /*     //matrixCopyCPU(&matrix[0][0], &result_matrix[0][0]);
70     matrixTransposeCPU(matrix, result_matrix, rows, columns)
           ;
71     printMatrix(matrix, rows, columns);
72     std::cout <<
           "-------------------------------------------------------"
            << std::endl;
73     printMatrix(result_matrix, rows, columns); */
74
75
76 /*
```

```
77 ------------------------eind cpu
       -------------------------------------
78 */
79
80 /*
81 ----------------------begin gpu, Global memory
       ------------------------
82  */
83
84     //met threads kan hier beginnen, moet wss niet, gewoon
           cpu/gpu vgl.
85
86     // Allocate device memory
87     cudaMalloc(&d_matrix, size* sizeof(int));
88     cudaMalloc(&d_result_matrix, size* sizeof(int));
89
90     // Copy data from host to device
91     cudaMemcpy(d_matrix, matrix.data(), size * sizeof(int),
           cudaMemcpyHostToDevice);
92     //moet niet voor c denk ik want is toch lege array?
93     // Launch kernel (1 block of N threads)
94     cudaEvent_t start_cuda, stop_cuda;
95     cudaEventCreate(&start_cuda);
96     cudaEventCreate(&stop_cuda);
97     cudaEventRecord(start_cuda);
98     matrixCopyGPU<<<blocks, threads>>>(d_matrix,
           d_result_matrix, matrix.size());
99     cudaDeviceSynchronize();
100    cudaEventRecord(stop_cuda);
101    cudaEventSynchronize(stop_cuda);
102    float ms;
103    cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
104    std::cout << "tijd: "<<ms << std::endl;
105    // Copy result back to host
106    cudaMemcpy(result_matrix.data(), d_result_matrix, size *
           sizeof(int), cudaMemcpyDeviceToHost);
107    cudaFree(d_matrix);
108    cudaFree(d_result_matrix);
109    std::cout << std::endl << std::endl;
110    printMatrix(matrix, rows, columns);
111    std::cout << "
           ----------------------------------------------------
           " << std::endl;
112    printMatrix(result_matrix, rows, columns);
113
114 /*
115 ----------------------eind gpu, Global memory
       ------------------------
116  */
117
```

```
118     return 0;
119 }
```

5.2 Oefening 2

```
1  #include <iostream>
2  #include <cuda_runtime.h>
3  #include <vector>
4  #include <iomanip>
5  #include <chrono>
6
7  __constant__ int d_const_matrix[16384]; //max 64 KB binary
       of decimal, no error given :)
8  #define TILE_SIZE 16
9
10
11 __global__ void matrixMultiplicationGlobal(int *m1, int *m2,
       int *result, int rows, int columns)
12 {
13     int idx = threadIdx.x + blockIdx.x * blockDim.x;
14     int size = rows * rows;
15
16     if(idx < size)
17     {
18         int row = idx / rows;
19         int col = idx % rows;
20         int sum = 0;
21
22         for(int k = 0; k < rows; k++)
23         {
24             sum += m1[row * rows + k] * m2[k * rows + col];
25         }
26
27         result[row * rows + col] = sum;
28     }
29 }
30
31
32 __global__ void matrixMultiplicationshared(int *m1, int *m2,
       int *result, int rows)
33 {
34     //shared memory, can be read by threads in the same
           block
35     //in contrast with global memory which can be read
           across blocks
36     //shared memory should be faster
37     __shared__ int tile_m1[TILE_SIZE][TILE_SIZE];
38     __shared__ int tile_m2[TILE_SIZE][TILE_SIZE];
```

```
39
40      int row = blockIdx.x * TILE_SIZE + threadIdx.x;
41      int col = blockIdx.y * TILE_SIZE + threadIdx.y;
42      int sum = 0;
43
44      // loop over tiles
45      for(int t = 0; t < (rows + TILE_SIZE - 1) / TILE_SIZE; t
            ++)
46      {
47          // 1. load tile from m1 into shared memory
48          // m1: slide across the row     threadIdx.y picks
                the column within the tile
49          if(row < rows && (t * TILE_SIZE + threadIdx.y) <
                rows){
50              tile_m1[threadIdx.x][threadIdx.y] = m1[row *
                    rows + (t * TILE_SIZE + threadIdx.y)];
51          }
52          else{
53              tile_m1[threadIdx.x][threadIdx.y] = 0;
54          }
55          // 2. load tile from m2 into shared memory
56          // m2: slide down the column     threadIdx.x picks
                the row within the tile
57          if(col < rows && (t * TILE_SIZE + threadIdx.x) <
                rows){
58              tile_m2[threadIdx.x][threadIdx.y] = m2[(t *
                    TILE_SIZE + threadIdx.x) * rows + col];
59          }
60          else{
61              tile_m2[threadIdx.x][threadIdx.y] = 0;
62          }
63          __syncthreads();  // wait for everyone
64          // 4. compute partial sum using shared memory
65          for(int k = 0; k < TILE_SIZE; k++){
66              sum += tile_m1[threadIdx.x][k] * tile_m2[k][
                    threadIdx.y];
67          }
68          __syncthreads();
69          //result
70      }
71
72      // write result
73      if(row < rows && col < rows)
74          result[row * rows + col] = sum;
75  }
76  __global__ void matrixMultiplicationsConstant(int *m1, int *
        result, int rows) {
77      int idx = threadIdx.x + blockIdx.x * blockDim.x;
78      int size = rows * rows;
79
```

```
80      if(idx < size)
81      {
82          int row = idx / rows;
83          int col = idx % rows;
84          int sum = 0;
85
86          for(int k = 0; k < rows; k++)
87          {
88              sum += m1[row * rows + k] * d_const_matrix[k *
                    rows + col];
89          }
90          result[row * rows + col] = sum;
91      }
92  }
93
94  void printMatrix(const std::vector<int>& m, int rows, int
        cols) {
95      for(int r = 0; r < rows; r++) {
96          for(int c = 0; c < cols; c++) {
97              std::cout << m[r * cols + c] << "␣";
98          }
99          std::cout << std::endl;
100     }
101 }
102 std::vector<int> generateMatrix(int rows, int cols) {
103     std::vector<int> matrix(rows * cols);
104     for(int i = 0; i < rows * cols; i++)
105         matrix[i] = rand() % 100;  //
106     return matrix;
107 }
108
109 std::vector<int> matrixMultiplicationCPU(const std::vector<
        int>& m1,
110                                          const std::vector<
                                                int>& m2,
111                                         const int rows){
112     //is return vector faster or pass result matrix by value
            ?
113     std::vector<int> result(rows*rows);
114     const int columns = rows; //alleen vierkanten, aanpassen
            indien rechthoek!
115     int idx;
116     int sum;
117     for(int r = 0; r < rows; r++) {
118         for(int c = 0; c < columns; c++) {
119             //voor iedere rij in m1 moeten we elke kolom
                    eens doorlopen.
120             //bij m1 kolom, rij vermenigvuldigen met
121             //m2 omgekeerde van m1 idx
122             sum = 0;
```

```
123             for(int k = 0; k<rows; k++){
124                 sum += m1[r * rows + k] * m2[k * columns + c
                        ];
125             }
126         idx = r * rows + c;
127         result[idx] = sum;
128         }
129     }
130     return result;
131 }
132
133 int main() {
134     float ms;
135
136     //std::vector<int> sizes = {3};//voor test of het werkt
137     std::vector<int> sizes = {16, 32, 64, 127, 128, 129,
            256, 512, 1024};//amount of rows/columns dimensions
            would be to "verwarrend"
138     srand(42); //wat is de vraag tho?
139     int* d_matrix1; //d_ = device = gpu
140     int* d_matrix2;
141     int* d_result_matrix;
142
143     int iterations = 100;
144
145     int threads1D = 256;
146     int blocks1D; // afh van size, dus in de loop waarde
147
148     cudaEvent_t start_cuda, stop_cuda;
149     cudaEventCreate(&start_cuda);
150     cudaEventCreate(&stop_cuda);
151
152
153     std::cout<<std::endl;
154     std::cout << std::setw(10) << "size"
155           << std::setw(20) << "global␣(ms)"
156           << std::setw(20) << "shared␣(ms)"
157           << std::setw(20) << "constant␣(ms)"
158           << std::setw(20) << "cpu␣(ms)" << std::endl;
159     std::cout << std::string(90, '-') << std::endl;
160     for(int size: sizes){
161         blocks1D = (size * size + threads1D - 1) / threads1D
                ;
162         std::vector<int> m1 = generateMatrix(size, size);
163         std::vector<int> m2 = generateMatrix(size, size);
164         std::vector<int> result_matrix(size * size);
165
166         std::cout << std::setw(10) << size;
167
168         //gpu klaarmaken
```

```
169         // Allocate device memory
170         //nodig voor alle benchmarks:
171         cudaMalloc(&d_matrix1, size * size* sizeof(int));
172         cudaMalloc(&d_result_matrix, size * size* sizeof(int
                ));
173         cudaMemcpy(d_matrix1, m1.data(), size * size *
                sizeof(int), cudaMemcpyHostToDevice);
174         //global memory:
175         //m2 wordt doorgegeven zoals m1
176         cudaMalloc(&d_matrix2, size * size* sizeof(int));
177         cudaMemcpy(d_matrix2, m2.data(), size * size *
                sizeof(int), cudaMemcpyHostToDevice);
178
179         cudaEventRecord(start_cuda);
180         for(int i = 0; i< iterations; i++){
181         matrixMultiplicationGlobal<<<blocks1D, threads1D>>>(
                d_matrix1, d_matrix2, d_result_matrix, size, size
                );
182         }
183         cudaDeviceSynchronize();//timen buiten de loop,
                anders deze overhead mee in resultaten
184         cudaEventRecord(stop_cuda);
185         cudaEventSynchronize(stop_cuda);
186         cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
187         float globalTime = ms/iterations;
188         std::cout << std::setw(20) << globalTime;
189
190
191         //shared memory:
192         //matrix 2 wordt meegegeven zoals global memory
193         //maar nu is dim anders
194         dim3 threads(TILE_SIZE, TILE_SIZE);  // 16x16 = 256
                threads per block
195         dim3 blocks((size + TILE_SIZE - 1) / TILE_SIZE, (
                size + TILE_SIZE - 1) / TILE_SIZE);
196
197         cudaEventRecord(start_cuda);
198         for(int i = 0; i< iterations; i++){
199             matrixMultiplicationshared<<<blocks, threads>>>(
                    d_matrix1,d_matrix2, d_result_matrix, size);
200         }
201         cudaDeviceSynchronize();//timen buiten de loop,
                anders deze overhead mee in resultaten
202         cudaEventRecord(stop_cuda);
203         cudaEventSynchronize(stop_cuda);
204         cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
205         float sharedTime = ms/iterations;
206         std::cout    << std::setw(20) << sharedTime;
207
208         //constant memory:
```

```
209         float constResult;
210         cudaError_t err = cudaMemcpyToSymbol(d_const_matrix,
                m2.data(), size * size * sizeof(int));//merk op,
                ToSymbol want cnst
211         if(err != cudaSuccess){
212             //std::cout << "CUDA error: " <<
                    cudaGetErrorString(err) << std::endl;
213             //continue;
                  constResult = -1;//prints N/A later
214         }
215         else{
216             cudaEventRecord(start_cuda);
217             for(int i = 0; i< iterations; i++){
218             matrixMultiplicationsConstant<<<blocks1D,
                    threads1D>>>(d_matrix1, d_result_matrix, size
                    );
219             }
220             cudaDeviceSynchronize();//timen buiten de loop,
                    anders deze overhead mee in resultaten
221             cudaEventRecord(stop_cuda);
222             cudaEventSynchronize(stop_cuda);
223             cudaEventElapsedTime(&ms, start_cuda, stop_cuda)
                    ;
224             constResult = (ms/iterations);
225         }
226         if(constResult > 0 ){
227               std::cout << std::setw(20) << constResult;}
228         else{
229             std::cout << std::setw(20) << "N/A";
230         }
231         std::cout << std::flush; //cpu duurt soms lang
232         //CPU:
233         const auto start = std::chrono::steady_clock::now();
234         for(int i = 0; i<iterations;i++){
235             matrixMultiplicationCPU(m1, m2, size);
236         }
237         const auto end = std::chrono::steady_clock::now();
238         const std::chrono::duration<double, std::milli>
                elapsed_miliseconds{end-start};
239         float CPUTime = elapsed_miliseconds.count()/
                iterations;
240
241         std::cout<< std::setw(20)<< CPUTime << std::endl;
242         // Copy result back to host, not really neceserry
                now
243         cudaMemcpy(result_matrix.data(), d_result_matrix,
                size*size * sizeof(int), cudaMemcpyDeviceToHost);
244         cudaFree(d_matrix1);
245
246     }
```

```
248     return 0;
249 }
```